

CITED REFERENCES AND FURTHER READING:

- Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §3.4 – §3.5.
- Dahlquist, G., and Björck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), §7.4.1 – §7.4.2.
- Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), §4.10–2.

4.4 Improper Integrals

For our present purposes, an integral will be “improper” if it has any of the following problems:

- its integrand goes to a finite limiting value at finite upper and lower limits, but cannot be evaluated *right on* one of those limits (e.g., $\sin x/x$ at $x = 0$)
- its upper limit is ∞ , or its lower limit is $-\infty$
- it has an integrable singularity at either limit (e.g., $x^{-1/2}$ at $x = 0$)
- it has an integrable singularity at a known place between its upper and lower limits
- it has an integrable singularity at an unknown place between its upper and lower limits

If an integral is infinite (e.g., $\int_1^\infty x^{-1} dx$), or does not exist in a limiting sense (e.g., $\int_{-\infty}^\infty \cos x dx$), we do not call it improper; we call it impossible. No amount of clever algorithmics will return a meaningful answer to an ill-posed problem.

In this section we will generalize the techniques of the preceding two sections to cover the first four problems on the above list. A more advanced discussion of quadrature with integrable singularities occurs in Chapter 19, notably §19.3. The fifth problem, singularity at an unknown location, can really only be handled by the use of a variable stepsize differential equation integration routine, as will be given in Chapter 17, or an adaptive quadrature routine such as in §4.7.

We need a workhorse like the extended trapezoidal rule (equation 4.1.11), but one that is an *open* formula in the sense of §4.1, i.e., does not require the integrand to be evaluated at the endpoints. Equation (4.1.19), the extended midpoint rule, is the best choice. The reason is that (4.1.19) shares with (4.1.11) the “deep” property of having an error series that is entirely even in h . Indeed there is a formula, not as well known as it ought to be, called the *Second Euler-Maclaurin summation formula*,

$$\begin{aligned}
 \int_{x_0}^{x_{N-1}} f(x) dx &= h[f_{1/2} + f_{3/2} + f_{5/2} + \cdots + f_{N-5/2} + f_{N-3/2}] \\
 &+ \frac{B_2 h^2}{4} (f'_{N-1} - f'_0) + \cdots \\
 &+ \frac{B_{2k} h^{2k}}{(2k)!} (1 - 2^{-2k+1}) (f_{N-1}^{(2k-1)} - f_0^{(2k-1)}) + \cdots
 \end{aligned} \tag{4.4.1}$$

This equation can be derived by writing out (4.2.1) with stepsize h , then writing it out again with stepsize $h/2$, and then subtracting the first from twice the second.

It is not possible to double the number of steps in the extended midpoint rule and still have the benefit of previous function evaluations (try it!). However, it is possible to *triple* the number of steps and do so. Shall we do this, or double and accept the loss? On the average, tripling does a factor $\sqrt{3}$ of unnecessary work, since the “right” number of steps for a desired accuracy criterion may in fact fall anywhere in the logarithmic interval implied by tripling. For doubling, the factor is only $\sqrt{2}$, but we lose an extra factor of 2 in being unable to use all the previous evaluations. Since $1.732 < 2 \times 1.414$, it is better to triple.

Here is the resulting structure, which is directly comparable to Trapzd.

quadrature.h

```
template <class T>
struct Midpnt : Quadrature {
    Routine implementing the extended midpoint rule.
    Doub a,b,s;                      Limits of integration and current value of inte-
    T &func;                          gral.
    Midpnt(T &func, const Doub aa, const Doub bb) :
        funk(func), a(aa), b(bb) {n=0;}
        The constructor takes as inputs func, the function or functor to be integrated between
        limits a and b, also input.
    Doub next(){
        Returns the nth stage of refinement of the extended midpoint rule. On the first call (n=1),
        the routine returns the crudest estimate of  $\int_a^b f(x)dx$ . Subsequent calls set n=2,3,... and
        improve the accuracy by adding  $(2/3) \times 3^{n-1}$  additional interior points.
        Int it,j;
        Doub x,tnm,sum,del,ddel;
        n++;
        if (n == 1) {
            return (s=(b-a)*func(0.5*(a+b)));
        } else {
            for(it=1,j=1;j<n-1;j++) it *= 3;
            tnm=it;
            del=(b-a)/(3.0*tnm);
            ddel=del+del;
            x=a+0.5*del;
            sum=0.0;
            for (j=0;j<it;j++) {
                sum += func(x);
                x += ddel;
                sum += func(x);
                x += del;
            }
            s=(s+(b-a)*sum/tnm)/3.0;
            return s;
        }
    }
    virtual Doub func(const Doub x) {return funk(x);} Identity mapping.
};
```

You may have spotted a seemingly unnecessary extra level of indirection in Midpnt, namely its calling the user-supplied function funk through an identity function func. The reason for this is that we are going to use mappings other than the identity mapping between funk and func to solve the problems of improper integrals listed above. The new quadratures will simply be derived from Midpnt with func overridden.

The structure Midpnt could be used to exactly replace Trapzd in a driver routine like qtrap (§4.2); one could simply change Trapzd<T> t(func,a,b) to Midpnt<T> t(func,a,b), and perhaps also decrease the parameter JMAX since

$3^{J_{\text{MAX}}-1}$ (from step tripling) is a much larger number than $2^{J_{\text{MAX}}-1}$ (step doubling). The open formula implementation analogous to Simpson's rule (qsimp in §4.2) could also substitute Midpnt for Trapzd, decreasing JMAX as above, but now also changing the extrapolation step to be

```
s=(9.0*st-ost)/8.0;
```

since, when the number of steps is tripled, the error decreases to 1/9th its size, not 1/4th as with step doubling.

Either the thus modified qtrap or qsimp will fix the first problem on the list at the beginning of this section. More sophisticated, and allowing us to fix more problems, is to generalize Romberg integration in like manner:

```
template<class T>                                                                    romberg.h
Doub qromo(Midpnt<T> &q, const Doub eps=3.0e-9) {
Romberg integration on an open interval. Returns the integral of a function using any specified
elementary quadrature algorithm q and Romberg's method. Normally q will be an open formula,
not evaluating the function at the endpoints. It is assumed that q triples the number of steps
on each call, and that its error series contains only even powers of the number of steps. The
routines midpnt, midinf, midsq1, midsqu, midexp are possible choices for q. The constants
below have the same meanings as in qromb.
    const Int JMAX=14, JMAXP=JMAX+1, K=5;
    VecDoub h(JMAXP),s(JMAX);
    Poly_interp polint(h,s,K);
    h[0]=1.0;
    for (Int j=1;j<=JMAX;j++) {
        s[j-1]=q.next();
        if (j >= K) {
            Doub ss=polint.rawinterp(j-K,0.0);
            if (abs(polint.dy) <= eps*abs(ss)) return ss;
        }
        h[j]=h[j-1]/9.0;          This is where the assumption of step tripling and an even
    }                             error series is used.
    throw("Too many steps in routine qromo");
}
```

Notice that we now pass a Midpnt object instead of the user function and limits of integration. There is a good reason for this, as we will see below. It does, however, mean that you have to bind things together before calling qromo, something like this, where we integrate from a to b:

```
Midpnt<Ftor> q(ftor,a,b);
Doub integral=qromo(q);
```

or, for a bare function,

```
Midpnt<Doub(Doub)> q(fbare,a,b);
Doub integral=qromo(q);
```

Laid back C++ compilers will let you condense these to

```
Doub integral = qromo(Midpnt<Ftor>(Ftor()),a,b));
```

or

```
Doub integral = qromo(Midpnt<Doub(Doub)>(fbare,a,b));
```

but uptight compilers may object to the way that a temporary is passed by reference, in which case use the two-line forms above.

As we shall now see, the function qromo, with its peculiar interface, is an excellent driver routine for solving all the other problems of improper integrals in our first list (except the intractable fifth).

The basic trick for improper integrals is to make a change of variables to eliminate the singularity or to map an infinite range of integration to a finite one. For example, the identity

$$\int_a^b f(x)dx = \int_{1/b}^{1/a} \frac{1}{t^2} f\left(\frac{1}{t}\right) dt \quad ab > 0 \quad (4.4.2)$$

can be used with *either* $b \rightarrow \infty$ and a positive, *or* with $a \rightarrow -\infty$ and b negative, and works for any function that decreases toward infinity faster than $1/x^2$.

You can make the change of variable implied by (4.4.2) either analytically and then use, e.g., `qromo` and `Midpnt` to do the numerical evaluation, *or* you can let the numerical algorithm make the change of variable for you. We prefer the latter method as being more transparent to the user. To implement equation (4.4.2) we simply write a modified version of `Midpnt`, called `Midinf`, which allows b to be infinite (or, more precisely, a very large number on your particular machine, such as 1×10^{99}), or a to be negative and infinite. Since all the machinery is already in place in `Midpnt`, we write `Midinf` as a derived class and simply override the mapping function.

```
template <class T>
struct Midinf : Midpnt<T>{
This routine is an exact replacement for midpnt, i.e., returns the nth stage of refinement of the
integral of funcc from aa to bb, except that the function is evaluated at evenly spaced points in
1/x rather than in x. This allows the upper limit bb to be as large and positive as the computer
allows, or the lower limit aa to be as large and negative, but not both. aa and bb must have
the same sign.
    Doub func(const Doub x) {
        return Midpnt<T>::funk(1.0/x)/(x*x);      Effect the change of variable.
    }
    Midinf(T &funcc, const Doub aa, const Doub bb) :
        Midpnt<T>(funcc, aa, bb) {
            Midpnt<T>::a=1.0/bb;                    Set the limits of integration.
            Midpnt<T>::b=1.0/aa;
        }
};
```

An integral from 2 to ∞ , for example, might be calculated by

```
Midinf<Ftor> q(ftor,2.,1.e99);
Doub integral=qromo(q);
```

If you need to integrate from a negative lower limit to positive infinity, you do this by breaking the integral into two pieces at some positive value, for example,

```
Midpnt<Ftor> q1(ftor,-5.,2.);
Midinf<Ftor> q2(ftor,2.,1.e99);
integral=qromo(q1)+qromo(q2);
```

Where should you choose the breakpoint? At a sufficiently large positive value so that the function `funk` is at least beginning to approach its asymptotic decrease to zero value at infinity. The polynomial extrapolation implicit in the second call to `qromo` deals with a polynomial in $1/x$, not in x .

To deal with an integral that has an integrable power-law singularity at its lower limit, one also makes a change of variable. If the integrand diverges as $(x - a)^{-\gamma}$, $0 \leq \gamma < 1$, near $x = a$, use the identity

$$\int_a^b f(x)dx = \frac{1}{1-\gamma} \int_0^{(b-a)^{1-\gamma}} t^{\frac{\gamma}{1-\gamma}} f\left(t^{\frac{1}{1-\gamma}} + a\right) dt \quad (b > a) \quad (4.4.3)$$

If the singularity is at the upper limit, use the identity

$$\int_a^b f(x)dx = \frac{1}{1-\gamma} \int_0^{(b-a)^{1-\gamma}} t^{\frac{\gamma}{1-\gamma}} f(b-t^{\frac{1}{1-\gamma}})dt \quad (b > a) \quad (4.4.4)$$

If there is a singularity at both limits, divide the integral at an interior breakpoint as in the example above.

Equations (4.4.3) and (4.4.4) are particularly simple in the case of inverse square-root singularities, a case that occurs frequently in practice:

$$\int_a^b f(x)dx = \int_0^{\sqrt{b-a}} 2tf(a+t^2)dt \quad (b > a) \quad (4.4.5)$$

for a singularity at a , and

$$\int_a^b f(x)dx = \int_0^{\sqrt{b-a}} 2tf(b-t^2)dt \quad (b > a) \quad (4.4.6)$$

for a singularity at b . Once again, we can implement these changes of variable transparently to the user by defining substitute routines for `Midpnt` that make the change of variable automatically:

```
template <class T> quadrature.h
struct Midsql : Midpnt<T>{
This routine is an exact replacement for midpnt, except that it allows for an inverse square-root
singularity in the integrand at the lower limit aa.
    Doub aorig;
    Doub func(const Doub x) {
        return 2.0*x*Midpnt<T>::funk(aorig+x*x);    Effect the change of variable.
    }
    Midsql(T &funcc, const Doub aa, const Doub bb) :
        Midpnt<T>(funcc, aa, bb), aorig(aa) {
        Midpnt<T>::a=0;
        Midpnt<T>::b=sqrt(bb-aa);
    }
};
```

Similarly,

```
template <class T> quadrature.h
struct Midsqu : Midpnt<T>{
This routine is an exact replacement for midpnt, except that it allows for an inverse square-root
singularity in the integrand at the upper limit bb.
    Doub borig;
    Doub func(const Doub x) {
        return 2.0*x*Midpnt<T>::funk(borig-x*x);    Effect the change of variable.
    }
    Midsqu(T &funcc, const Doub aa, const Doub bb) :
        Midpnt<T>(funcc, aa, bb), borig(bb) {
        Midpnt<T>::a=0;
        Midpnt<T>::b=sqrt(bb-aa);
    }
};
```